

Recursive Functions

DM580: Functional Programming

Casper Bach

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 6: Recursive Functions

Recursion vs. Loops

Recursion vs. Loops

- Loops work well for linear traversals

Recursion vs. Loops

- Loops work well for linear traversals
- Recursive functions are simpler for structured data and non-linear traversals

Recursion vs. Loops

- Loops work well for linear traversals
- Recursive functions are simpler for structured data and non-linear traversals
- Recursive functions require a *call stack*; loops do not which makes them cheaper
 - However, *tail-recursive functions* can be optimized to make them as cheap as loops

Recursion vs. Loops

- Loops work well for linear traversals
- Recursive functions are simpler for structured data and non-linear traversals
- Recursive functions require a *call stack*; loops do not which makes them cheaper
 - However, *tail-recursive functions* can be optimized to make them as cheap as loops
- Any loop can be easily transformed into a recursive function
 - The other direction (recursive function → loop) is possible too, but less easy

Recursion vs. Loops

- Loops work well for linear traversals
- Recursive functions are simpler for structured data and non-linear traversals
- Recursive functions require a *call stack*; loops do not which makes them cheaper
 - However, *tail-recursive functions* can be optimized to make them as cheap as loops
- Any loop can be easily transformed into a recursive function
 - The other direction (recursive function $\rightarrow$ loop) is possible too, but less easy
- Recursive functions tend to match the structure of inductive proofs $\Rightarrow$ recursive functions tend to be easy to prove correct

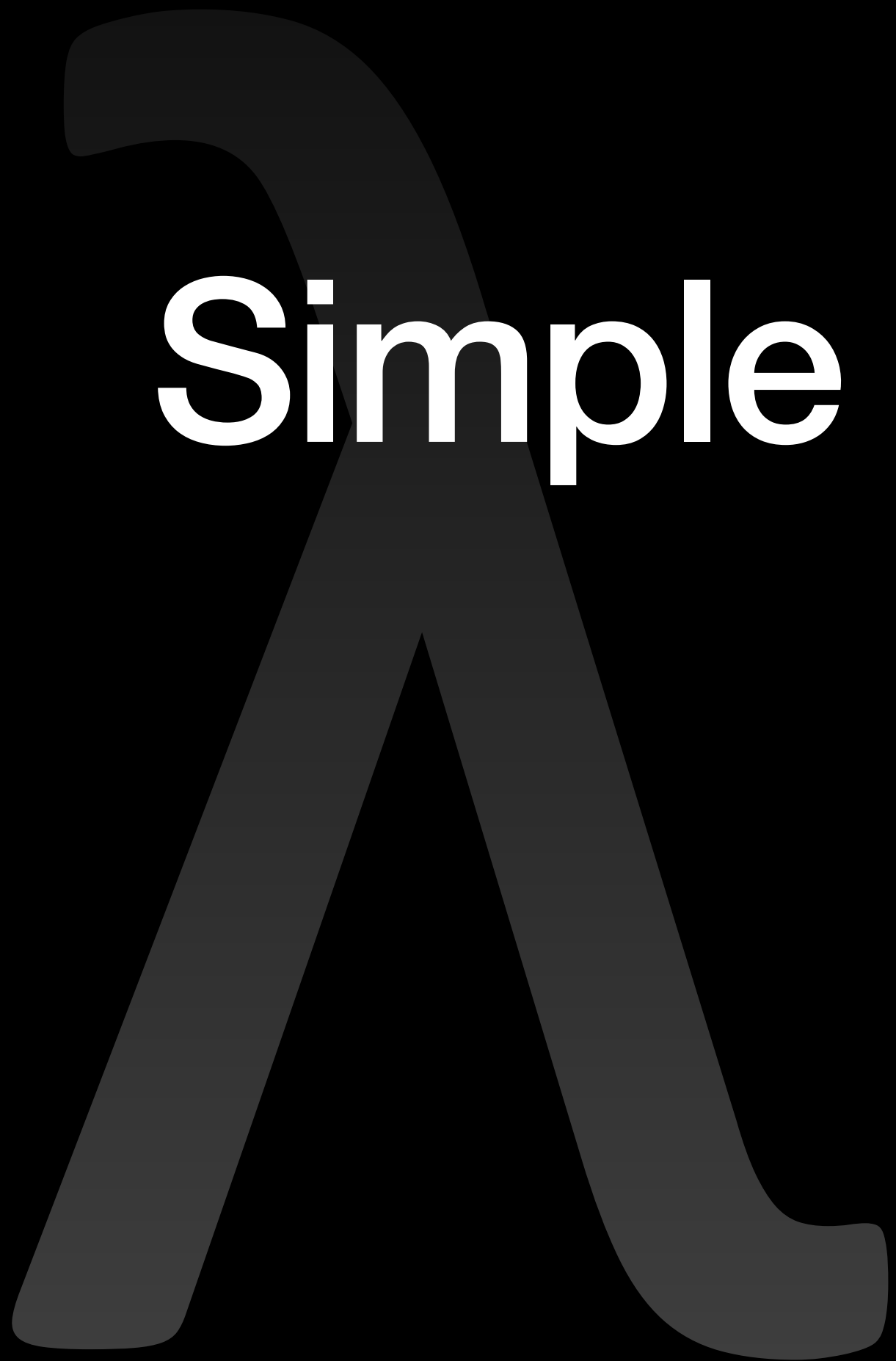
Recursion vs. Loops

- Loops work well for linear traversals
- Recursive functions are simpler for structured data and non-linear traversals
- Recursive functions require a *call stack*; loops do not which makes them cheaper
 - However, *tail-recursive functions* can be optimized to make them as cheap as loops
- Any loop can be easily transformed into a recursive function
 - The other direction (recursive function $\rightarrow$ loop) is possible too, but less easy
- Recursive functions tend to match the structure of inductive proofs $\Rightarrow$ recursive functions tend to be easy to prove correct
- Haskell does not have primitive support for loops; only recursion

Learning Objectives

- Explain what a recursive function is
- Write simple programs that use recursion
- Use equational rewriting to expand function definitions
- Explain what a tail call optimization is
- Write simple, tail-recursive programs

Simple Recursive Functions



What is a Recursive Function?

- A function that calls itself
- E.g.,

```
fac :: Int → Int
fac n = if n ≤ 1 then 1 else n * fac (n-1)
```

- We will look at examples of recursive functions that traverse structured data other than numbers and lists next week
- This week, to keep matters simple, we focus on recursive functions for numbers and lists

Another Recursive Function

Summing Integers

```
sum :: [Int] → Int
sum []      = 0
sum (x:xs)  = x + sum xs
```

Exercise 1 on Recursive Functions

What Does This Function Do?

```
foo :: [Int] → [Int]
foo []      = []
foo (x:xs)  = x:foo xs
```

Exercise 2 on Recursive Functions

What Does This Function Do?

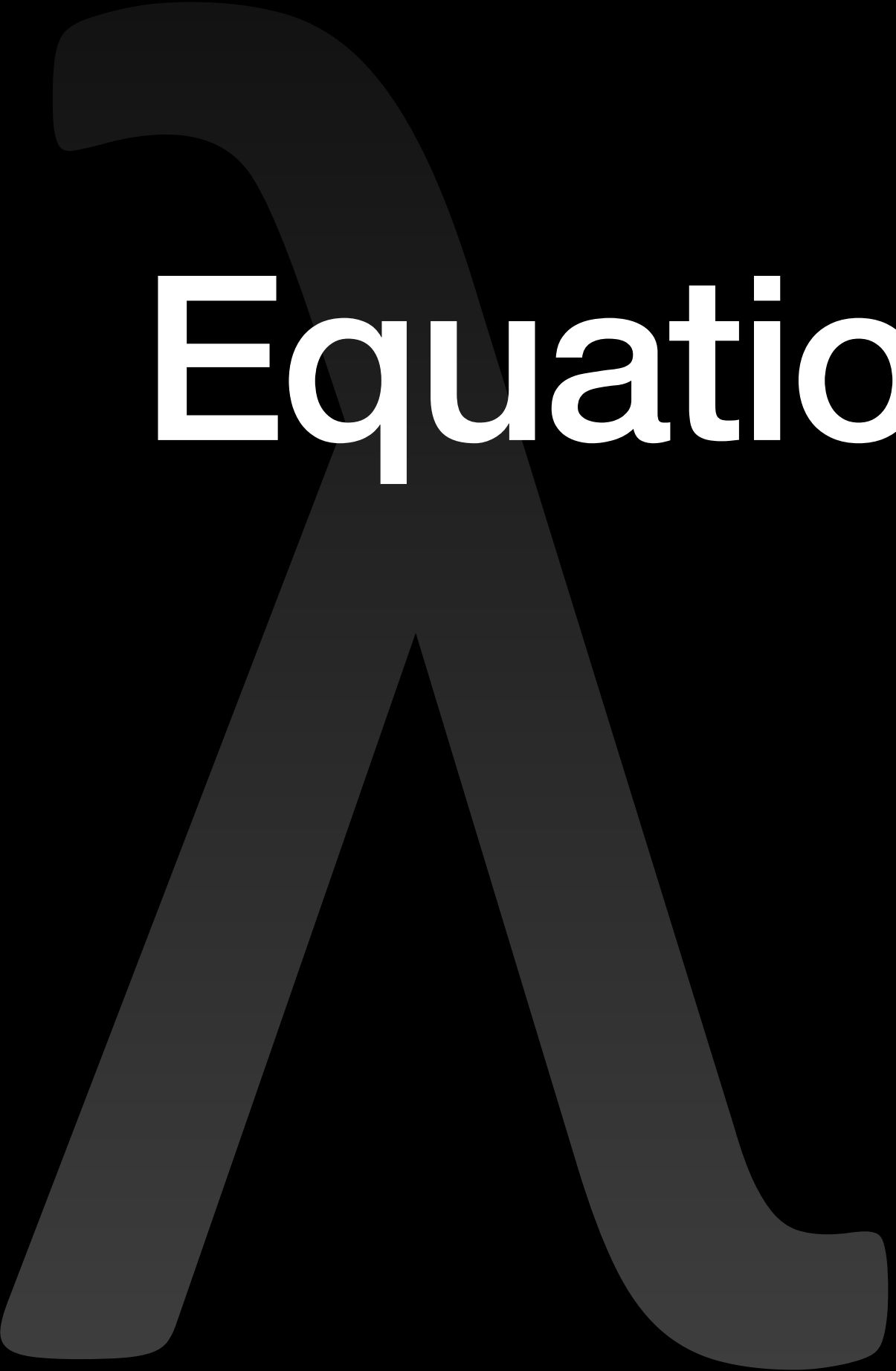
```
bar :: [Int] → [Int]
bar []      = []
bar (x:xs) = bar xs ++ [x]
```

Exercise 3 on Recursive Functions

Find Remainder After

- Implement a function `remainderAfter` which takes an element `x` and a list `ys` as input, and finds the first occurrence of `x` in `ys`, and returns the tail of the list at that point.

```
remainderAfter :: Eq a => a -> [a] -> [a]
remainderAfter x []           = _
remainderAfter x (y:ys)      = _
```



Equational Rewriting

Aside: Function Declarations and Equational Rewriting

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

`1337 * fac 2 `div` 19`

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19  
= {unfold `fac 2`}
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

`1337 * fac 2 `div` 19`

`= {unfold `fac 2`}`

`1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19`

`= {simplify}`

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
= {unfold `fac n`}
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
= {unfold `fac n`}
1337 * (2 * (if 1 ≤ 1 then 1 else fac (1-1))) `div` 19
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
= {unfold `fac n`}
1337 * (2 * (if 1 ≤ 1 then 1 else fac (1-1))) `div` 19
= {simplify}
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
= {unfold `fac n`}
1337 * (2 * (if 1 ≤ 1 then 1 else fac (1-1))) `div` 19
= {simplify}
1337 * (2 * 1) `div` 19
```

Aside: Function Declarations and Equational Rewriting

- A benefit of functional programming is that we can simplify and reason using simple equational rewriting
- For example, *any* `fac x`, for some integer `x`, in *any* program context, can be replaced by the right-hand side of `fac n`

```
1337 * fac 2 `div` 19
= {unfold `fac 2`}
1337 * (if 2 ≤ 1 then 1 else 2 * fac (2-1)) `div` 19
= {simplify}
1337 * (2 * fac 1) `div` 19
= {unfold `fac n`}
1337 * (2 * (if 1 ≤ 1 then 1 else fac (1-1))) `div` 19
= {simplify}
1337 * (2 * 1) `div` 19
= 140
```



Tail Call Optimization

Reminder from Earlier

- Recursive functions require a *call stack*; loops do not which makes them cheaper
 - However, *tail-recursive functions* can be optimized to make them as cheap as loops

What is Tail Recursion?

- A function is tail recursive when its recursive call is the very last thing it does
- That is, the result of the recursive call is returned without further computation
- For such calls, no state needs to be kept on the stack for the return trip
- *Tail call optimization*: transforms a tail-recursive function into a loop

Is This Function Tail Recursive?

A function is tail recursive when its recursive call is the very last thing it does

```
fac :: Int → Int
fac n = if n ≤ 1
      then 1
      else n * fac (n-1)
```

Is This Function Tail Recursive?

A function is tail recursive when its recursive call is the very last thing it does

```
fac :: Int → Int
fac n = if n ≤ 1
      then 1
      else n * fac (n-1)
```



No

Is This Function Tail Recursive?

A function is tail recursive when its recursive call is the very last thing it does

```
facAcc :: Int → Int
facAcc n a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

Is This Function Tail Recursive?

A function is tail recursive when its recursive call is the very last thing it does

```
facAcc :: Int → Int
facAcc n a = if n ≤ 1
             then a
             else facAcc (n-1) (n*a)
```

Yes.

**Branching on a value does not
require stack allocation.**

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

```
facAcc :: Integer → Integer
facAcc !n !a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

```
facAcc :: Integer → Integer
facAcc !n !a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

```
main :: IO ()
main = do
  putStrLn (show (facAcc 10000 1))
  putStrLn (show (fac 10000))
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

```
facAcc :: Integer → Integer
facAcc !n !a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

```
$ ghc -rtsopts Fac.hs
$ ./Fac +RTS -K10k
```

```
main :: IO ()
main = do
  putStrLn (show (facAcc 10000 1))
  putStrLn (show (fac 10000))
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

```
facAcc :: Integer → Integer
facAcc !n !a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

```
main :: IO ()
main = do
  putStrLn (show (facAcc 10000 1))
  putStrLn (show (fac 10000))
```

```
$ ghc -rtsopts Fac.hs
$ ./Fac +RTS -K10k
```

[first prints a big number]

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Ensure that the program eagerly evaluates arguments, by annotating patterns with a ! (bang pattern). Without it, lazy evaluation may cause thunk allocation (more in week 48).
- Use Integer (unbounded integers) instead of Int (64 bit integers) for large factorials
- Run our program with +RTS -K10k flag to limit stack size

```
facAcc :: Integer → Integer
facAcc !n !a = if n ≤ 1
              then a
              else facAcc (n-1) (n*a)
```

```
main :: IO ()
main = do
  putStrLn (show (facAcc 10000 1))
  putStrLn (show (fac 10000))
```

```
$ ghc -rtsopts Fac.hs
$ ./Fac +RTS -K10k
```

[first prints a big number]

[then crashes due to stack overflow]

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Comparing performance of an alternative test program

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Comparing performance of an alternative test program

```
$ ghc SumAcc.hs
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Comparing performance of an alternative test program

```
$ ghc SumAcc.hs
```

```
$ ./SumAcc
```

```
./SumAcc 0.95s user  
          0.01s system  
          99% cpu  
          0.969 total
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Comparing performance of an alternative test program

```
$ ghc SumAcc.hs
```

```
$ ./SumAcc
```

```
./SumAcc 0.95s user  
          0.01s system  
          99% cpu  
          0.969 total
```

```
$ ghc SumStack.hs
```

Tail Call Optimization Experiment

To validate that our programs are TCO'ed (tail-call-optimized)

- Comparing performance of an alternative test program

```
$ ghc SumAcc.hs
```

```
$ ./SumAcc
```

```
./SumAcc 0.95s user  
          0.01s system  
          99% cpu  
          0.969 total
```

```
$ ghc SumStack.hs
```

```
$ ./SumStack
```

```
./SumStack 4.33s user  
           0.40s system  
           99% cpu  
           4.738 total
```